

# How to Use This SQL Guide

Candidate reports mention 3 SQL questions in the online assessment, and SQL is a central topic in the technical interview.

Each problem below provides the table schema with sample data, the question, a solution query, the expected result, and an explanation.

Every query has been executed in SQLite and verified to produce the expected output.

## SQL Problems

### Problem 1: Second Highest Salary

*Difficulty: Easy | Pattern: Subquery / DISTINCT*

#### Schema & Sample Data

```
CREATE TABLE employee (id INTEGER, name TEXT, salary INTEGER);
INSERT INTO employee VALUES (1, 'Asha', 50000), (2, 'Ravi', 80000), (3, 'Meera', 80000), (4, 'John', 60000)
```

#### Question

From the employee table, find the second highest distinct salary. If it does not exist, return NULL.

#### Solution Query

```
SELECT MAX(salary) AS second_highest
FROM employee
WHERE salary < (SELECT MAX(salary) FROM employee);
```

#### Expected Output

```
second_highest
60000
```

#### Explanation

The inner query finds the top salary (80000). The outer query takes the maximum salary strictly below it, giving the second highest distinct value (60000). Using MAX with a '<' filter naturally ignores duplicate top salaries.

### Problem 2: Count Employees per Department

*Difficulty: Easy | Pattern: GROUP BY / Aggregation*

#### Schema & Sample Data

```
CREATE TABLE emp (id INTEGER, name TEXT, dept TEXT);
INSERT INTO emp VALUES (1, 'A', 'Sales'), (2, 'B', 'Sales'), (3, 'C', 'Tech'), (4, 'D', 'Tech'), (5, 'E', 'Tech')
```

#### Question

Count the number of employees in each department and order the result by employee count descending, then department name ascending.

#### Solution Query

```
SELECT dept, COUNT(*) AS emp_count
```

```
FROM emp
GROUP BY dept
ORDER BY emp_count DESC, dept ASC;
```

### Expected Output

```
dept|emp_count
Tech|3
Sales|2
HR|1
```

### Explanation

GROUP BY collapses rows per department; COUNT(\*) counts members. Ordering by count descending puts the largest team first, with ties broken alphabetically.

## Problem 3: Employees Earning Above Their Department Average

*Difficulty: Medium | Pattern: Correlated Subquery*

### Schema & Sample Data

```
CREATE TABLE staff (id INTEGER, name TEXT, dept TEXT, salary INTEGER);
INSERT INTO staff VALUES (1, 'A', 'Sales', 40000), (2, 'B', 'Sales', 60000), (3, 'C', 'Tech', 90000), (4, 'D', 'Tech', 90000);
```

### Question

List the names (and salaries) of employees who earn more than the average salary of their own department. Order by name.

### Solution Query

```
SELECT name, salary
FROM staff s
WHERE salary > (SELECT AVG(salary) FROM staff
                WHERE dept = s.dept)
ORDER BY name;
```

### Expected Output

```
name|salary
B|60000
C|90000
```

### Explanation

The correlated subquery computes the average salary for each employee's own department (referenced by s.dept). Sales avg = 50000 (B qualifies); Tech avg = 70000 (C qualifies, D is equal so excluded).

## Problem 4: Find Duplicate Emails

*Difficulty: Easy | Pattern: GROUP BY + HAVING*

### Schema & Sample Data

```
CREATE TABLE person (id INTEGER, email TEXT);
INSERT INTO person VALUES (1, 'a@x.com'), (2, 'b@x.com'), (3, 'a@x.com'), (4, 'c@x.com'), (5, 'b@x.com');
```

### Question

Find all email addresses that appear more than once, along with how many times they occur. Order by email.

### Solution Query

```
SELECT email, COUNT(*) AS occurrences
```

```
FROM person
GROUP BY email
HAVING COUNT(*) > 1
ORDER BY email;
```

### Expected Output

```
email|occurrences
a@x.com|3
b@x.com|2
```

### Explanation

GROUP BY email groups identical addresses; HAVING filters groups after aggregation to keep only those with a count greater than one. WHERE cannot be used here because the condition is on an aggregate.

## Problem 5: Customers With No Orders

---

*Difficulty: Medium | Pattern: LEFT JOIN / Anti-Join*

### Schema & Sample Data

```
CREATE TABLE customers (id INTEGER, name TEXT);
CREATE TABLE orders (order_id INTEGER, customer_id INTEGER);
INSERT INTO customers VALUES (1, 'Alice'), (2, 'Bob'), (3, 'Carol'), (4, 'Dan');
INSERT INTO orders VALUES (101, 1), (102, 1), (103, 3);
```

### Question

List the names of customers who have never placed an order. Order by name.

### Solution Query

```
SELECT c.name
FROM customers c
LEFT JOIN orders o ON c.id = o.customer_id
WHERE o.order_id IS NULL
ORDER BY c.name;
```

### Expected Output

```
name
Bob
Dan
```

### Explanation

A LEFT JOIN keeps every customer; unmatched customers get NULLs for order columns. Filtering WHERE o.order\_id IS NULL keeps exactly those with no matching orders (Bob and Dan).

## Problem 6: Running Total of Daily Sales

---

*Difficulty: Medium | Pattern: Window Function (SUM OVER)*

### Schema & Sample Data

```
CREATE TABLE sales (day INTEGER, amount INTEGER);
INSERT INTO sales VALUES (1, 100), (2, 150), (3, 50), (4, 200);
```

### Question

For each day, compute the cumulative (running) total of sales up to and including that day, ordered by day.

### Solution Query

```
SELECT day, amount,
       SUM(amount) OVER (ORDER BY day) AS running_total
FROM sales
ORDER BY day;
```

### Expected Output

```
day|amount|running_total
1|100|100
2|150|250
3|50|300
4|200|500
```

### Explanation

The window function `SUM(amount) OVER (ORDER BY day)` adds up amounts from the first row through the current row, producing a running total without collapsing rows.

## Problem 7: Top 2 Products by Sales in Each Category

---

*Difficulty: Hard | Pattern: Window Function (RANK / PARTITION BY)*

### Schema & Sample Data

```
CREATE TABLE product_sales (category TEXT, product TEXT, sales INTEGER);
INSERT INTO product_sales VALUES ('A', 'p1', 100), ('A', 'p2', 200), ('A', 'p3', 150), ('B', 'p4', 300), ('B', 'p6', 400);
```

### Question

For each category, return the top 2 products by sales. Order by category, then sales descending.

### Solution Query

```
SELECT category, product, sales
FROM (
    SELECT category, product, sales,
           RANK() OVER (PARTITION BY category
                        ORDER BY sales DESC) AS rnk
    FROM product_sales
) ranked
WHERE rnk <= 2
ORDER BY category, sales DESC;
```

### Expected Output

```
category|product|sales
A|p2|200
A|p3|150
B|p6|400
B|p4|300
```

### Explanation

`RANK()` with `PARTITION BY category` restarts the ranking for each category, ordering products by sales descending. The outer query keeps ranks 1 and 2, giving the top two products per category.

# Tredence

---

## SQL Problems

Applied Role(s):

Analyst - Data Scientist

Online Assessment Preparation Material

Version 1.0 • Last Updated: July 2026

Compiled from verified previous-year patterns, community-reported questions and research-backed company-style practice.

# Table of Contents

|                                                                       |   |
|-----------------------------------------------------------------------|---|
| How to Use This SQL Guide                                             | 1 |
| SQL Problems                                                          | 1 |
| Problem 1: Second Highest Salary . . . . .                            | 1 |
| Problem 2: Count Employees per Department . . . . .                   | 1 |
| Problem 3: Employees Earning Above Their Department Average . . . . . | 2 |
| Problem 4: Find Duplicate Emails . . . . .                            | 2 |
| Problem 5: Customers With No Orders . . . . .                         | 3 |
| Problem 6: Running Total of Daily Sales . . . . .                     | 3 |
| Problem 7: Top 2 Products by Sales in Each Category . . . . .         | 4 |